

딥러닝 성능 최적화 요약 (8장)

1. 데이터를 사용한 성능 최적화

1-1. 최대한 많은 데이터 수집하기

일반적으로 딥러닝·머신러닝 알고리즘은 데이터양이 많을수록 성능이 좋아진다. 딥러닝은 데이터가 어느 수준 이상 쌓여야 머신러닝 알고리즘을 추월하는 성능을 발휘하기 때문에, 가능한 한 많은 빅데이터를 확보하는 것이 첫 번째 최적화 전략이다.

1-2. 데이터 생성하기

충분한 데이터를 수집할 수 없는 상황이라면 기존 데이터를 변형·증강(augmentation)하여 인위적으로 데이터를 늘리는 방법을 사용한다. 이미지 데이터의 경우 회전, 반전, 크기 조정, 밝기 조절 등의 조작을 통해 데이터셋을 확장할 수 있다.

1-3. 데이터 범위(Scale) 조정하기

활성화 함수의 종류에 따라 입력 데이터의 범위를 맞춰 주는 것이 중요하다.

- **시그모이드(Sigmoid)** 사용 시: 데이터 범위를 0~1로 조정
- **하이퍼볼릭 탄젠트(tanh)** 사용 시: 데이터 범위를 -1~1로 조정

이 외에도 정규화(Normalization), 규제화(Regularization), 표준화(Standardization)를 적용하면 모델이 더 빠르고 안정적으로 학습할 수 있다.

2. 알고리즘을 이용한 성능 최적화

머신러닝·딥러닝 알고리즘은 매우 다양하며, 처음 선택한 알고리즘이 최적이지 아닐 수 있다. 따라서 유사한 용도의 여러 알고리즘을 직접 훈련시켜보고 성능을 비교한 뒤 최적의 알고리즘을 선택해야 한다.

- **분류 문제:** SVM, K-최근접 이웃(KNN) 등 비교
- **시계열 문제:** RNN, LSTM, GRU 등 비교

단순히 논문이나 이론에 의존하는 것이 아니라, 실제 데이터에 맞게 실험을 반복하는 것이 핵심이다.

3. 알고리즘 튜닝을 통한 성능 최적화

성능 최적화에서 가장 많은 시간이 소요되는 단계다. 하나의 모델을 선택한 뒤 다양한 하이퍼파라미터를 변경하면서 최적 조합을 찾아야 하며, 수십~수백 번의 반복 훈련이 필요할 수 있다.

3-1. 진단 (Diagnosis)

모델 성능이 어느 순간 멈췄다면, 학습 곡선(accuracy/loss 그래프)을 통해 원인을 분석해야 한다.

상황	원인	해결책
훈련 성능 >> 검증 성능	과적합(Overfitting)	규제화, 드롭아웃 적용
훈련·검증 모두 성능 낮음	과소적합(Underfitting)	네트워크 구조 변경, 에포크 수 증가
특정 시점 이후 검증 성능 하락	과적합 시작	조기 종료 고려

3-2. 가중치 초기화 (Weight Initialization)

가중치 초기값은 작은 난수를 사용하는 것이 일반적이다. 적절한 초기값이 불분명한 경우, 오토인코더 같은 비지도 학습으로 사전 훈련(pre-training)을 진행한 후 지도 학습을 이어가는 방법도 있다.

3-3. 학습률 (Learning Rate)

- 네트워크 계층이 많을수록(깊을수록): 학습률을 높게 설정
- 네트워크 계층이 적을수록(얕을수록): 학습률을 낮게 설정
- 초기에 크거나 작은 난수 값을 설정하고 결과를 보며 조금씩 조정하는 방식이 일반적이다

3-4. 활성화 함수 (Activation Function)

활성화 함수를 변경할 경우 손실 함수도 함께 변경해야 하는 경우가 많다. 데이터 유형과 원하는 출력 형태를 정확히 이해하고 선택해야 한다.

- 은닉층: 시그모이드, 하이퍼볼릭 탄젠트, ReLU 등
- 출력층: 소프트맥스(다중 분류), 시그모이드(이진 분류)

3-5. 배치(Batch)와 에포크(EPOCH)

최근 딥러닝 트렌드는 **큰 에포크 + 작은 배치** 조합이다. 그러나 데이터셋 크기에 맞게 배치 크기를 조정하거나, 전체 데이터를 하나의 배치로 훈련해보는 등 다양한 테스트를 권장한다.

3-6. 옵티마이저(Optimizer)와 손실 함수(Loss Function)

- 기본: 확률적 경사 하강법(SGD)
- 대안: Adam, RMSProp 등도 좋은 성능을 보임
- 네트워크 구성에 따라 결과가 달라지므로 여러 조합을 실험 후 선택

3-7. 네트워크 구성 (Network Topology)

최적의 네트워크 구성은 정해진 공식이 없다. 다음 방식으로 실험해야 한다.

- **넓은 네트워크**: 은닉층 하나에 뉴런 수를 많이
- **깊은 네트워크**: 계층 수를 늘리되 뉴런 수는 줄임
- **혼합 방식**: 두 가지를 결합하여 최적 구성 탐색

4. 앙상블을 이용한 성능 최적화

두 개 이상의 모델을 결합(앙상블)하여 단일 모델보다 더 높은 성능을 끌어내는 방법이다. 다양한 알고리즘이나 하이퍼파라미터로 훈련된 여러 모델의 예측 결과를 합산·투표하는 방식으로 동작한다. 성능 향상 효과는 있으나, 모델 훈련 비용이 그만큼 증가한다.

5. 하드웨어(GPU)를 이용한 성능 최적화

5-1. CPU vs GPU 비교

구분	CPU	GPU
처리 방식	직렬(Sequential) 처리	병렬(Parallel) 처리
핵심 구성	ALU + 컨트롤 + 캐시	다수의 ALU (수백~수천 개)
코어 속도	빠름	상대적으로 느림
적합 작업	순차적 연산 (재귀 등)	행렬 연산, 역전파 등 병렬 연산
딥러닝 성능	낮음	높음 (CPU 5개 < GPU 1개)

GPU는 병렬 처리에 특화되어 있어, 수백만~수천만 건의 데이터를 벡터로 변환해 연산하는 딥러닝에 필수적이다. CPU로 며칠 걸릴 훈련을 GPU로는 몇 시간 내로 단축할 수 있다.

5-2. CUDA와 cuDNN

- **CUDA** (Compute Unified Device Architecture): NVIDIA에서 개발한 GPU 프로그래밍 툴. 대량 연산을 동시 처리 가능하게 해주며, CUDA 등장 이후 일반 개발자도 GPU 프로그래밍이 가능해졌다.
- **cuDNN**: CUDA 기반 딥러닝 특화 라이브러리. PyTorch, TensorFlow, Caffe 등 주요 프레임워크에서 활용된다.

5-3. PyTorch GPU 설정

```
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
model = model.to(device)
data = data.to(device)
```

GPU가 없는 환경에서도 CPU로 실행 가능하며, PyTorch 예제 대부분은 CPU/GPU 모두 지원한다.

6. 하이퍼파라미터를 이용한 심화 성능 최적화

6-1. 정규화 / 규제화 / 표준화 개념 정리

용어	설명	수식
정규화 (Normalization)	데이터 범위를 0~1로 조정 (MinMaxScaler)	$(x - x_{\min}) / (x_{\max} - x_{\min})$
규제화 (Regularization)	모델 복잡도를 줄이기 위해 필터(제약) 적용	드롭아웃, 조기 종료 등
표준화 (Standardization)	평균 0, 표준편차 1로 변환 (Z-score)	$(x - \mu) / \sigma$

6-2. 배치 정규화 (Batch Normalization)

2015년 논문 "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift"에서 제안된 기법이다.

문제: 신경망의 각 층마다 활성화 함수가 적용되면서 입력 값의 분포가 계속 변하는 **내부 공변량 변화(Internal Covariate Shift)** 발생 → 기울기 소멸(Gradient Vanishing) / 기울기 폭발(Gradient Exploding) 유발

해결: 미니배치 단위로 평균 0, 표준편차 1로 정규화하여 분포를 안정화

동작 과정:

1. 미니배치 평균(μ_B) 계산
2. 미니배치 분산(σ^2_B) 계산
3. 정규화: $\hat{x} = (x - \mu_B) / \sqrt{\sigma^2_B + \epsilon}$
4. 스케일·이동 조정: $y = \gamma\hat{x} + \beta$ (γ, β 는 학습 파라미터)

위치: 완전연결층(FC) 또는 합성곱층(Conv) 뒤, 활성화 함수 앞

```
# PyTorch 예시
self.classifier = nn.Sequential(
    nn.Linear(784, 48),
    nn.BatchNorm1d(48),    # 배치 정규화 삽입
    nn.ReLU(),
    nn.Linear(48, 24),
    nn.BatchNorm1d(24),
    nn.ReLU(),
    nn.Linear(24, 10)
)
```

효과: 배치 정규화 적용 모델이 미적용 모델에 비해 더 낮은 오차, 더 안정적인 수렴 확인

단점:

- 배치 크기가 너무 작으면 정규화 값이 불안정해짐 (분산이 0인 경우 정규화 불가)
- RNN에 적용 시 계층별로 별도 정규화가 필요해 모델이 복잡해짐

6-3. 드롭아웃 (Dropout)

개념: 훈련 중 매 스텝마다 일정 비율의 뉴런을 **무작위로 비활성화**하여 특정 뉴런에 과도하게 의존하는 것을 방지한다. 비활성화된 뉴런은 신호를 전달하지 않아 과적합을 억제한다.

- 훈련 시: 지정한 비율(예: 0.2)만큼 뉴런 무작위 제거
- 테스트 시: 모든 뉴런 사용 + 드롭아웃 비율을 곱해 출력 조정

```
# PyTorch 예시
model_dropout = torch.nn.Sequential(
    torch.nn.Linear(1, N_h),
    torch.nn.Dropout(0.2),    # 20% 뉴런 비활성화
    torch.nn.ReLU(),
    torch.nn.Linear(N_h, N_h),
    torch.nn.Dropout(0.2),
    torch.nn.ReLU(),
)
```

```
torch.nn.Linear(N_h, 1),
)
```

효과: 드롭아웃 미적용 모델은 시간이 지날수록 훈련 데이터에 과적합되어 경계 부근의 점들을 따라가는 반면, 드롭아웃 적용 모델은 일반화된 예측선을 유지한다.

단점: 훈련 시간이 길어짐. 그럼에도 과적합 방지 효과가 크기 때문에 실무에서 자주 사용된다.

6-4. 조기 종료 (Early Stopping)

개념: 검증 데이터셋의 오차(val_loss)가 더 이상 감소하지 않고 증가하기 시작하는 시점에 학습을 중단하는 규제 기법이다.

동작 원리:

- 과적합 발생 전: training loss ↓, validation loss ↓ (함께 감소)
- 과적합 발생 후: training loss ↓, validation loss ↑ (역전 발생)
- 조기 종료는 val_loss가 증가하기 시작하는 시점에 학습을 멈춤

주요 파라미터:

- **patience:** 개선 없이 기다릴 에포크 수 (너무 작으면 조기 종료가 너무 빠름)
- **delta:** 개선으로 인정할 최소 변화량
- **verbose:** 종료 과정 출력 여부

```
class EarlyStopping():
    def __init__(self, patience=5, delta=0):
        self.patience = patience
        self.counter = 0
        self.best_score = None
        self.early_stop = False
        self.delta = delta

    def __call__(self, val_loss, model):
        score = -val_loss
        if self.best_score is None:
            self.best_score = score
        elif score < self.best_score + self.delta:
            self.counter += 1
            if self.counter >= self.patience:
                self.early_stop = True
        else:
            self.best_score = score
            self.counter = 0
```

주의: 조기 종료는 최고 성능 모델을 보장하지 않는다. 성능 향상보다는 **자원(CPU/메모리) 효율화** 목적이 더 강하다. patience를 너무 작게 설정하면 모델이 충분히 학습하기 전에 종료될 수 있다.

6-5. 학습률 감소 (Learning Rate Scheduler)

학습률을 고정값으로 유지하는 대신, 학습 진행에 따라 점진적으로 줄여 최적화를 돕는 기법이다. PyTorch에서는 `ReduceLROnPlateau`를 사용한다.

동작: 지정한 `patience` 횟수만큼 `val_loss` 개선이 없으면, 학습률을 `factor`배로 감소

```
lr_scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer,
    mode='min',           # val_loss 기준 (최솟값 추구)
    patience=5,           # 5 에포크 개선 없으면 조정
    factor=0.5,           # 학습률 = 기존 학습률 × 0.5
    min_lr=1e-6,         # 학습률 하한선
    verbose=True
)
```

효과 비교 (실험 결과):

- 인수 없음: 정확도 그래프가 위아래로 크게 요동, 에포크 10 이후 `val_loss` 우상향 (과적합 조짐)
- 학습률 감소 적용: 정확도 곡선이 완만해지고 `val_loss` 우상향 현상 소멸, 최종 검증 정확도 향상
- 조기 종료 적용: 빠른 훈련 종료로 자원 절약, 단 안정성은 낮을 수 있음

7. 사전 훈련 모델(Transfer Learning) 활용

직접 네트워크를 설계하는 대신 **ImageNet**으로 사전 훈련된 **ResNet50** 같은 모델을 불러와 파인튜닝하는 방법도 성능 향상에 효과적이다.

```
model = models.resnet50(pretrained=True)

# 특성 추출만 할 경우 파라미터 고정
for param in model.parameters():
    param.requires_grad = False

# 마지막 분류 계층만 교체하여 학습
model.fc = nn.Linear(2048, 2) # 이진 분류 예시
```

- 전체 파라미터 수: 약 25,557,032개
- 사전 훈련 모델 활용 시 적은 데이터로도 높은 성능 달성 가능

핵심 정리

기법	목적	효과
데이터 증강	데이터 부족 해결	일반화 성능 향상
배치 정규화	내부 공변량 변화 해결	학습 속도↑, 안정성↑
드롭아웃	과적합 방지	일반화 성능↑

기법	목적	효과
조기 종료	과적합 방지	자원 효율화
학습률 감소	최적화 미세 조정	수렴 안정성↑
GPU 활용	연산 병렬화	훈련 시간 대폭 단축
앙상블	여러 모델 결합	예측 정확도↑

결론: 성능 최적화는 단일 기법으로 해결되지 않는다. 학습 곡선을 꼼꼼히 분석하고, 과적합·과소적합 여부와 수렴 패턴을 파악한 뒤 상황에 맞는 기법들을 조합해서 적용해야 한다.